

BoardScan

Chess Position Digitiser (Photo -> FEN -> Evaluation)

Computer Vision - Course Project Report

Name: [Your Name]

Roll No.: [Roll No.]

Course / Faculty: [Course]

Date: [Date]

Repository: <https://github.com/abhishek-k-33/boardscan>

2. Introduction

Over-the-board chess players must re-enter positions into an engine by hand, which is slow and error-prone. BoardScan photographs a physical board from an arbitrary angle and returns the position in FEN plus an engine evaluation. Board localisation uses classical CV only (edge detection, Hough transform, homography); a small CNN is used solely for per-square piece classification. The report documents requirements, design, implementation, honest evaluation results, and lessons learned.

3. Problem Statement

Manual re-entry of over-the-board positions into analysis engines is slow and error-prone. Target users: club players digitising games, coaches reviewing student games, tournament arbiters recording positions. Scope: one RGB photo of a full board (0-45 deg tilt, varied lighting) -> validated FEN + eval. Out of scope: video tracking, partial boards, clock/scoresheet OCR.

4. Functional Requirements

Module	Input -> Output
1. Detection & Rectification	RGB photo -> 800x800 top-down board
2. Square Segmentation & Classification	warped board -> 8x8 label grid (13 classes)
3. FEN Assembly & Engine Analysis	label grid + side-to-move -> FEN, cp eval, best move
4. Debug / Visualisation Layer	any stage -> annotated overlay images

5. Non-functional Requirements

NFR	Target / evidence
Performance	Full CLI pipeline 0.07-0.08 s on CPU (budget 3 s); downscale to 1200 px
Reliability	Every stage returns a typed ok/data/error result; UI shows errors, never crashes
Usability	Editable 8x8 grid: one wrong square never forces a retake
Maintainability	All CV params in config.py; pure independently-testable functions
Observability	--debug dumps numbered stage images to debug/
Resource efficiency	212k-param CNN, 829 KB fp32 (budget 5 MB), CPU-only

6. System Architecture

BoardScan — System Architecture

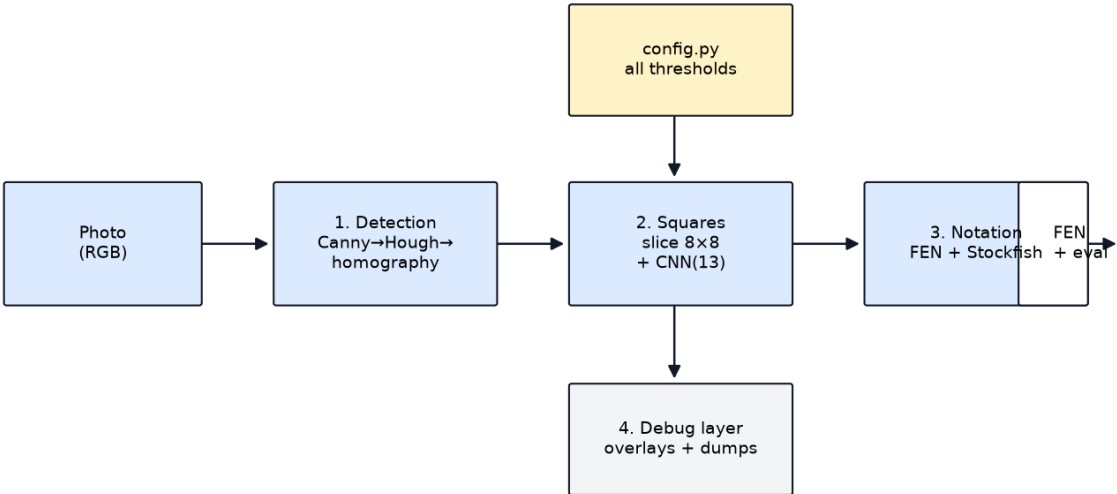


Fig. 1 - Module data flow: photo -> FEN + eval.

Photo -> preprocess (gray/CLAHE/bilateral) -> Canny/Hough/lines -> corners (lattice + contour fallback, larger quad wins) -> homography warp (800x800) -> biased 8x8 slicing -> PieceCNN -> orientation-aware FEN assembly (king-count validation) -> Stockfish or degraded FEN-only mode. config.py feeds every stage; the debug layer taps each one. Storage is file-based: data/raw photos + .fen sidecars, data/squares crops + manifests (no database; ER diagram not applicable).

7. Design Diagrams

BoardScan — Use Case Diagram

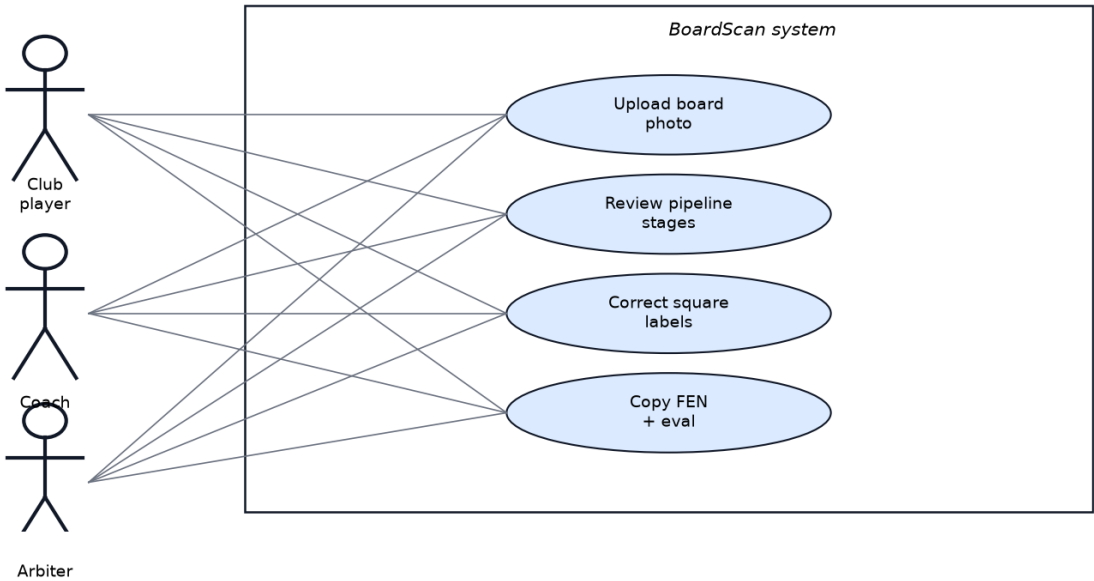


Fig. 2 - Use cases: upload, review stages, correct labels, copy FEN.

BoardScan — Sequence Diagram (upload → FEN)

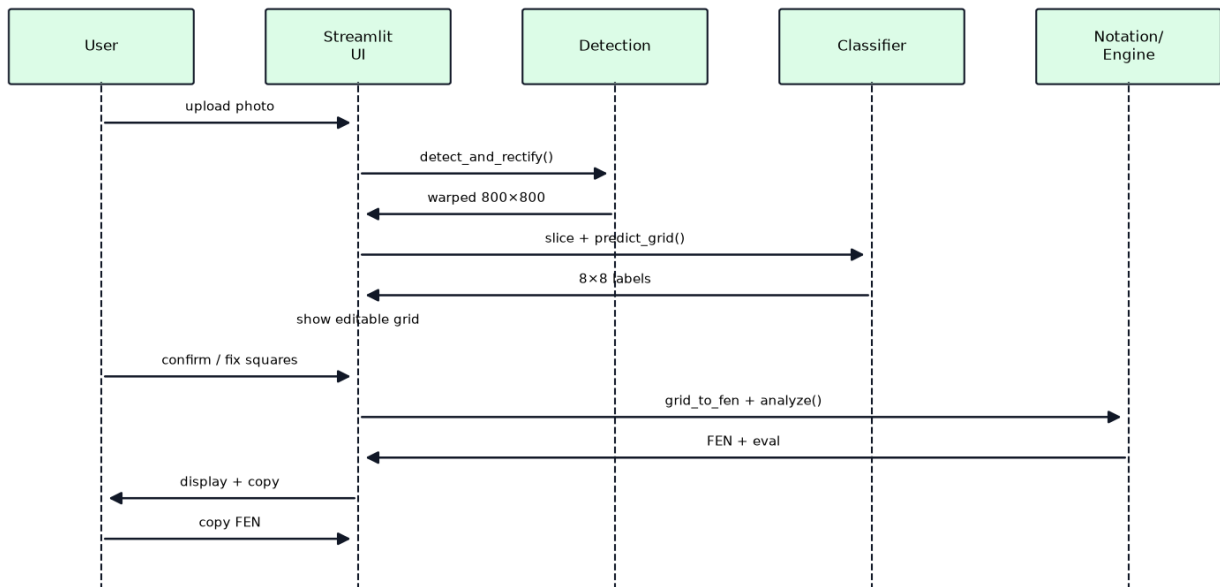


Fig. 3 - Upload-to-FEN sequence across UI and modules.

BoardScan — Class Diagram

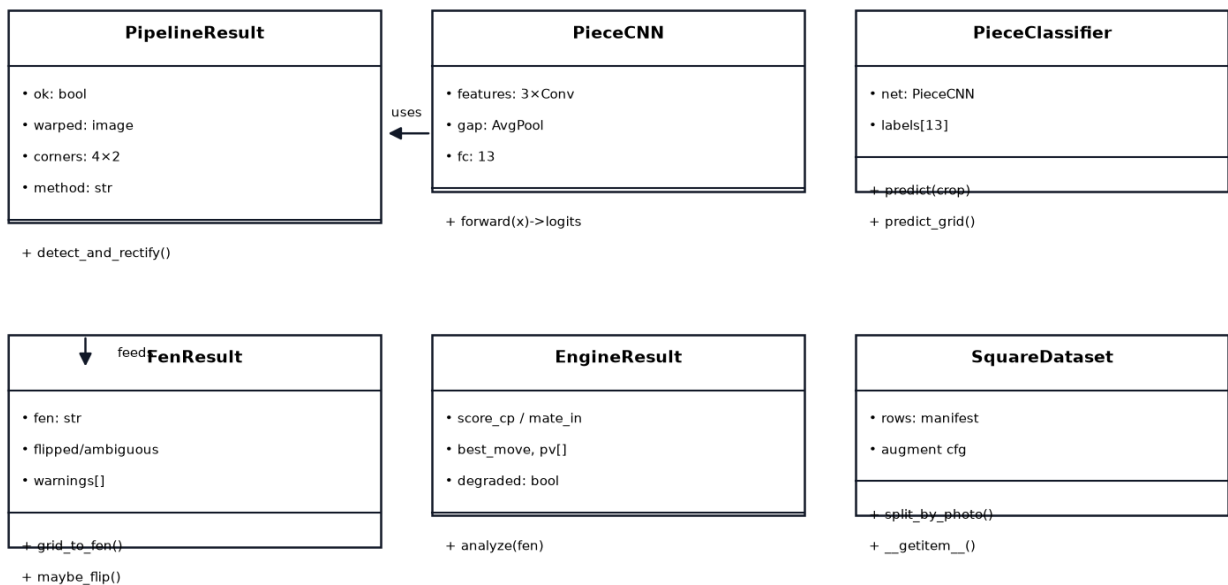


Fig. 4 - Key result/class containers and their relations.

Workflow (pipeline order): upload -> detect_and_rectify -> slice_board -> predict_grid -> grid_to_fen -> analyze -> display. Each arrow carries a typed result; any failure short-circuits to a user-facing message.

8. Design Decisions & Rationale

- Bilateral over Gaussian: preserves board edges while smoothing wood grain; one-flag ablation toggle.
- CLAHE over global equalisation: handles uneven lamp light across the board.
- Probabilistic Hough over standard Hough: segment endpoints enable angle clustering; minLineLength tied to square size.
- Lattice + contour fallback (larger quad wins): lattice extrapolates clipped corners; contour traces occluded borders.
- Per-square CNN over whole-board detector: 64 easy 13-class problems beat one 32-object detection at ~8k squares.
- Split by photo, not by square: same-photo squares share lighting; naive split inflates accuracy (we reproduced a fake 100%).
- King-count validation: >1 king per side is an error; missing kings (empty calibration boards) only warn.
- Mixed synthetic + realistic training: single-style training collapses on unseen art (train/serve skew).

9. Implementation Details

- Stack: Python, OpenCV primitives, NumPy, PyTorch (classifier only), python-chess, Streamlit, pytest.
- detection/: preprocess.py, lines.py, corners.py, rectify.py, pipeline.py - pure functions, config-driven.
- squares/: slicer.py (100x100 crops, +25 px upward bias for piece heads), labels.py (FEN <-> grid), classifier.py.
- model/: PieceCNN 48/96/192ch + GAP + FC(13); SquareDataset with brightness/contrast, +/-3 deg rot, +/-2 px shift; no hflip.
- notation/: fen.py (orientation score, auto-flip, ambiguous flag), engine.py (fixed-depth Stockfish, degraded mode).
- Training: 8572 squares (3900 synthetic + 4672 rendered-board), 70/15/15 by photo, Adam 1e-3 + StepLR, 96px input.
 - Data: honi05/chess-positions-cv sample (FEN in filename) imported to data/raw/NNN.jpg + NNN.fen; disclosed openly.

10. Screenshots / Results

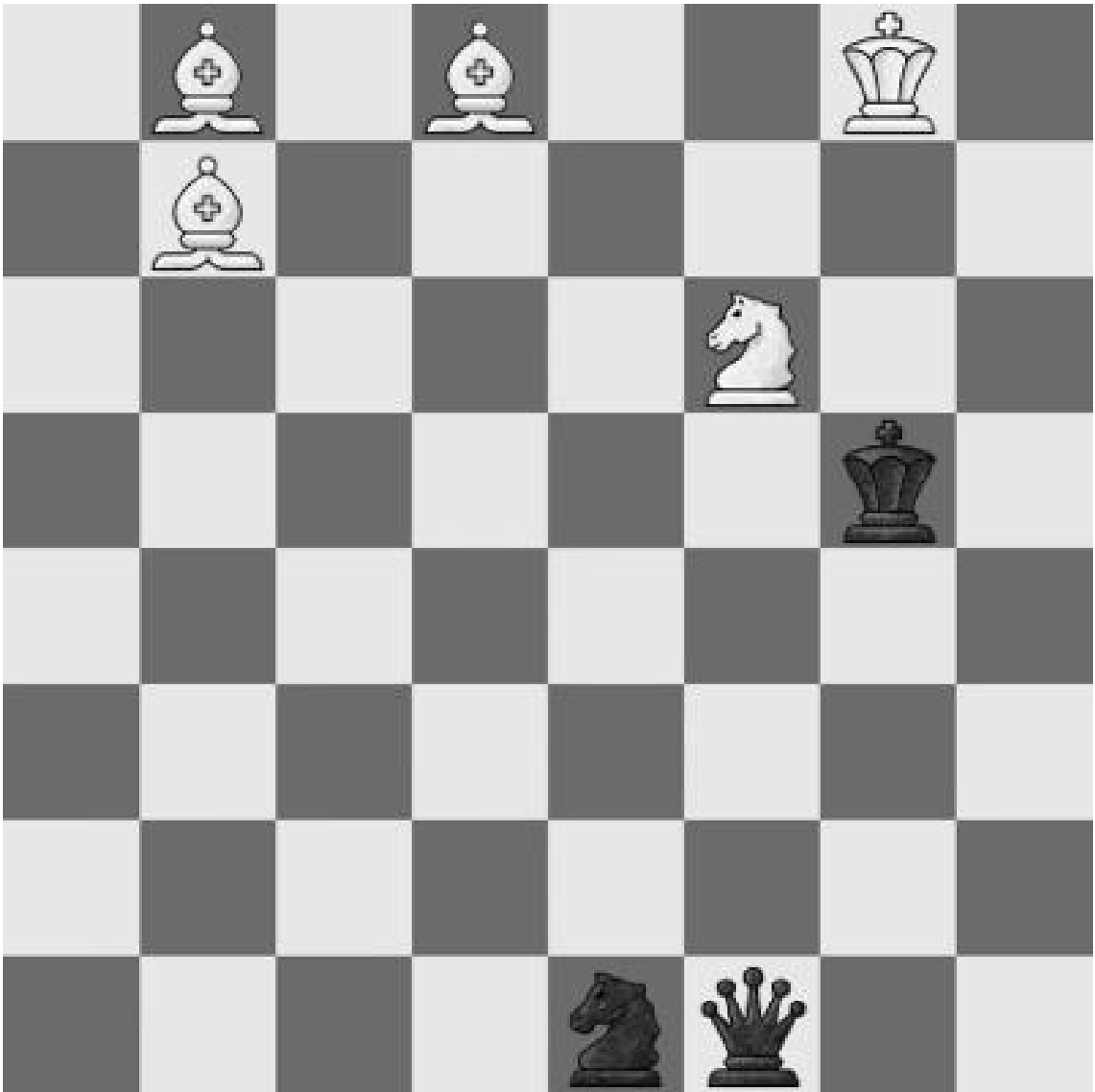


Fig. - Preprocessed input (01_gray.png)

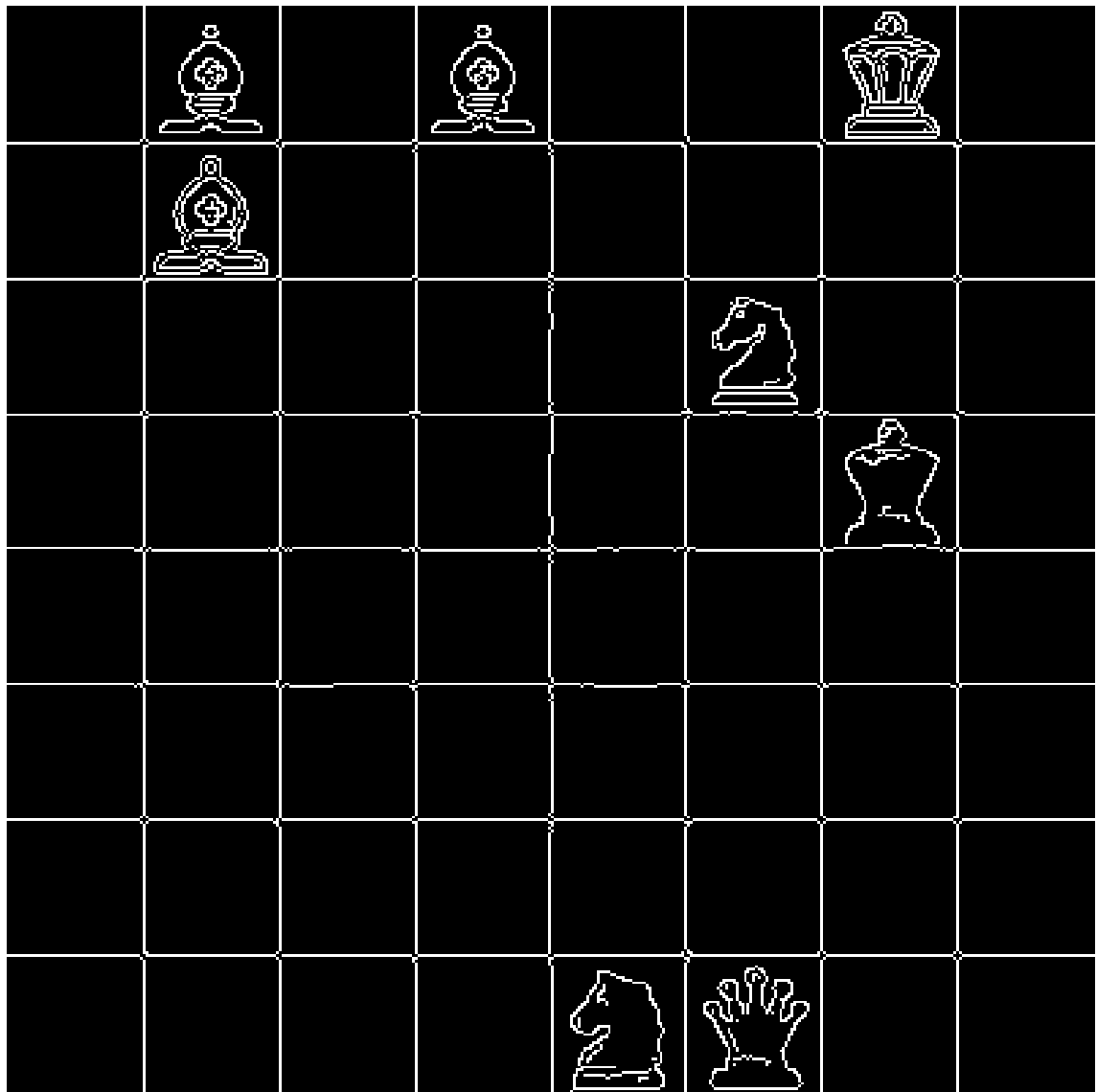


Fig. - Canny edge map (04_edges.png)



Fig. - Rectified 800x800 board (07_warped.png)

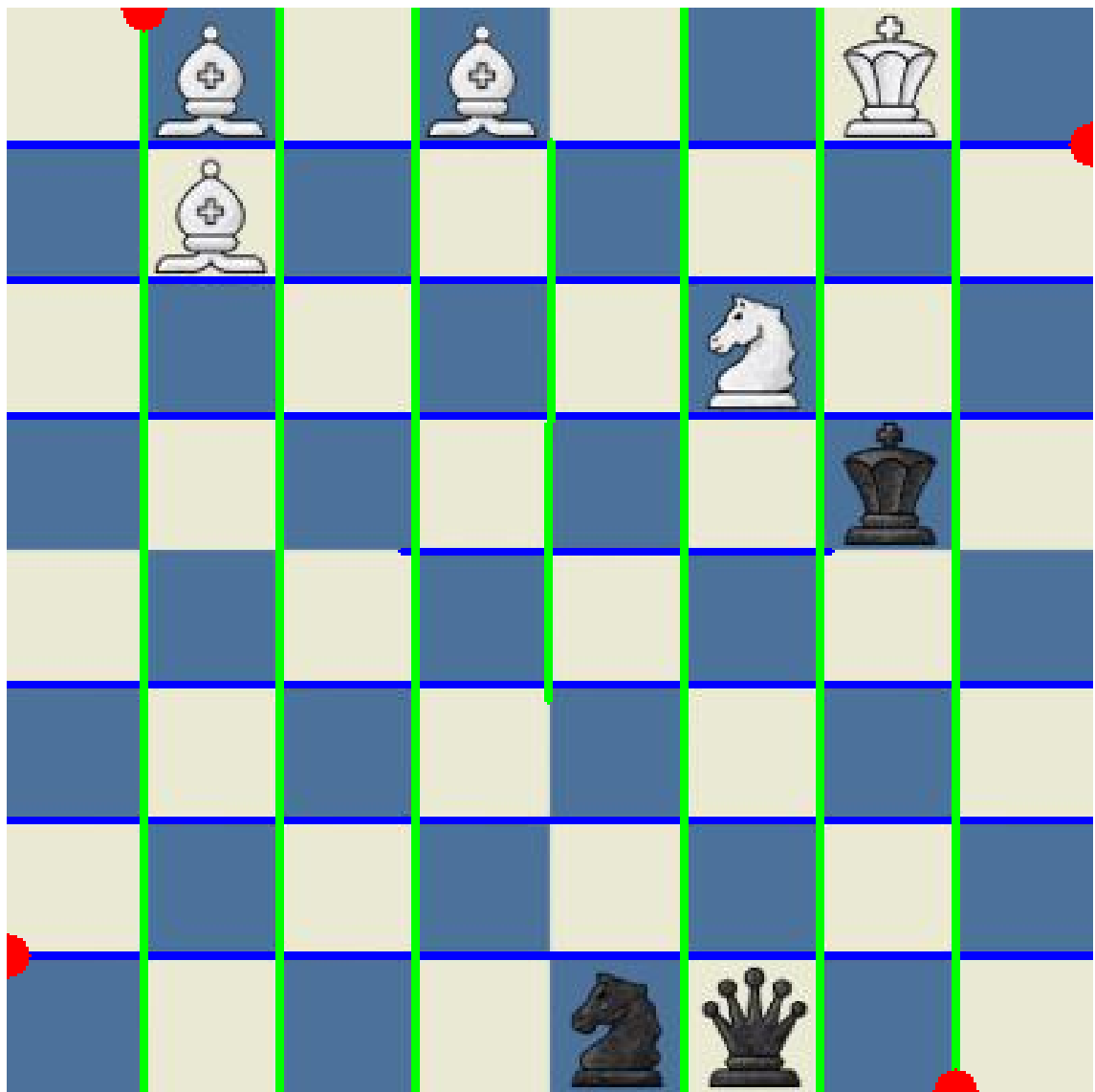


Fig. - Lines + corners overlay (08_overlay.png)

true \ pred

	empty	P	N	B	R	Q	K	p	n	b	r	q	k
empty	52	0	0	0	0	0	0	0	0	0	0	0	0
P	0	45	0	0	0	0	0	0	0	0	0	0	0
N	0	0	49	0	0	0	0	0	0	0	0	0	0
B	0	0	0	57	0	0	0	0	0	0	0	0	0
R	0	0	0	0	48	0	0	0	0	0	0	0	0
Q	0	0	0	0	0	56	0	0	0	0	0	0	0
K	0	0	0	0	0	0	43	0	0	0	0	0	0
p	0	0	0	0	0	0	0	41	0	0	0	0	0
n	0	0	0	0	0	0	0	0	47	0	0	0	0
b	0	0	0	0	0	0	0	0	0	52	0	0	0
r	0	0	0	0	0	0	0	0	0	0	44	0	0
q	0	0	0	0	0	0	0	0	0	0	0	56	0
k	0	0	0	0	0	0	0	0	0	0	0	0	50

Fig. - Confusion matrix, combined held-out set.

Measured results (honest)

Metric	Value
Rectify success (15 synthetic boards)	100%, grid within 10 px
Corner error mean / p95	1.5 px / 2.0 px
Classifier per-square (synthetic held-out)	100% (640 squares)
Classifier per-square (combined held-out)	89.4% (1344 squares)
Live demo, 5 rendered boards	53-60/64 squares each; 0/5 exact FEN
Full CLI time	~0.08 s CPU
Unit tests	16/16 pass

Preprocessing ablation (15 synthetic boards)

Setting	Success	Mean px	p95 px
CLAHE on + bilateral (shipped)	100%	1.5	2.0
CLAHE off + bilateral	100%	1.8	2.6
CLAHE on + gaussian	100%	1.2	1.7
CLAHE off + gaussian	100%	1.3	1.9

On clean synthetic boards every setting succeeds and differences are sub-pixel noise (Gaussian even edges ahead

slightly). The toggles exist so the same study can be repeated on real photos with lighting variation and wood grain, where CLAHE/bilateral are expected to matter - listed as future work.

11. Testing Approach

- test_rectify.py: Phase-1 acceptance on synthetic boards (grid tolerance 10 px, 90% bar).
- test_squares.py: slicer determinism/bias, FEN mapping, no-leakage split, 5 MB budget.
- test_fen.py: round-trips, orientation flip/tie, two-kings error, bad-input rejection.
- test_pipeline.py: composed-board exact-FEN smoke test; engine degraded/live modes.
- Acceptance mapping: detection 15/15, classifier 89.4% realistic (spec bar 95% - documented gap), e2e exact-FEN on synthetic positions pass, 0/5 on realistic sample (reported, not hidden).

12. Challenges Faced

- OpenCV 5 returns HoughLinesP as (N,4) not (N,1,4): handled both shapes.
- Dashed board-border edges missed by strict Hough: gap-tolerant tuning (minLen 60, gap 30).
- Lattice hull locked onto interior grid lines: larger-quad-wins against contour fixed it.
- Class-pure pseudo-photos faked 100% accuracy: mixed dealing exposed true 77%, then bolder shapes + mild aug + wider net recovered.
- Train/serve skew (wood-tone training vs grey boards): mixed realistic data closed most of it.
- Laptop too slow for training: moved to Colab GPU via checked-in notebook.
- Streamlit data_editor has no help kwarg in deployed version: replaced with caption.

13. Learnings & Key Takeaways

- Classical detection generalises across art styles for free; the CNN only knows what it was shown.
- Split discipline (by photo) and mixed-class photos are what make accuracy numbers believable.
- Exact-FEN is brutal: 89% per-square still gives 0/5 exact boards - hence human-in-the-loop correction.
- Typed per-stage results turn every failure into report evidence instead of a crash.

14. Future Enhancements

- Retrain on ChessReD real smartphone photos; repeat ablation on real lighting.
- Install Stockfish for live eval. Customise: side-to-move inference from move history.
- Mobile capture app; video-frame averaging for robustness.

15. References

- Masouris et al., End-to-End Chess Recognition + ChessReD, 4TU.ResearchData, DOI 10.4121/99b5c721-280b-450b-b058-b2900b69a90f.
- honi05/chess-positions-cv, Hugging Face datasets (rendered boards with FEN filenames).
- Fenit annotated chessboard dataset, Mendeley Data, DOI 10.17632/wpp9xnbc6.
- OpenCV docs (Canny, HoughLinesP, getPerspectiveTransform); python-chess; PyTorch; Streamlit.